

# Note technique

## Visualisation décisionnelle des soumissions Kobo dans G2M

### Objet

Cette note propose une analyse et une approche d'implémentation pour créer, dans G2M, une page HTML de visualisation décisionnelle des données collectées via KoboToolbox. L'objectif est de transformer une soumission brute Kobo, structurée selon les noms techniques du XLSForm, en une fiche métier lisible par un responsable de supervision.

Le module cible doit permettre d'afficher, pour chaque unité d'enquête, par exemple un site, l'ensemble des informations utiles sous une forme claire : rubriques métier, indicateurs, photos, localisation, commentaires, alertes et données de contrôle.

### Étape 1 - Problèmes et difficultés de la transposition de vue

#### 1. Correspondance entre noms techniques et libellés lisibles

Les soumissions Kobo utilisent les noms techniques définis dans la colonne `name` du XLSForm. Ces clés peuvent être peu compréhensibles : `modA/enqueteur`, `gps_site`, `q102_statut`, etc. Pour une fiche de pilotage, il faut retrouver les libellés métier issus des colonnes `label`, éventuellement selon la langue.

**Impact utilisateur** : sans traduction, la fiche reste technique et peu exploitable par un décideur.

**Impact maintenabilité** : si la correspondance est codée en dur dans les vues, chaque évolution du formulaire oblige à modifier le code applicatif.

#### 2. Gestion des groupes et des préfixes Kobo

Dans Kobo, les questions placées dans des groupes peuvent apparaître dans le JSON avec des chemins de type `groupe/question`. Ces préfixes doivent être interprétés pour reconstituer une organisation logique par rubriques.

**Impact utilisateur** : les informations peuvent sembler dispersées ou redondantes si la structure du formulaire n'est pas reconstruite.

**Impact maintenabilité** : il faut éviter que chaque groupe fasse l'objet d'un traitement spécifique dans le code.

#### 3. Gestion des groupes répétés

Les groupes répétés produisent plusieurs lignes de réponses pour une même section du formulaire : membres d'un ménage, équipements, observations multiples, photos multiples, etc. Leur rendu ne peut pas être identique à un champ simple.

**Impact utilisateur** : un affichage plat devient illisible et ne permet pas de comparer les éléments répétés.

**Impact maintenabilité** : les répétitions doivent être représentées par un modèle générique, par exemple sous forme de sous-tableaux ou de cartes répétées.

## 4. Typage des champs

Les champs Kobo ne doivent pas tous être affichés de la même façon. Un texte, un nombre, une date, une géolocalisation, une image, un choix multiple ou un booléen appellent des composants différents.

**Impact utilisateur** : une date brute, une coordonnée GPS ou une liste de choix non formatée dégrade fortement la lisibilité.

**Impact maintenabilité** : le moteur de rendu doit utiliser un type métier stable, idéalement dérivé du XLSForm ou d'un fichier de mapping.

## 5. Choix simples et choix multiples

Les réponses à des questions `select_one` ou `select_multiple` peuvent être stockées sous forme de codes. Il faut les convertir vers les libellés des choix, issus de la feuille `choices` du XLSForm.

**Impact utilisateur** : afficher `status_ok` ou `a_verifier` est moins parlant que "Validé" ou "À vérifier".

**Impact maintenabilité** : la table des choix doit être centralisée, sinon les libellés risquent de diverger entre les écrans.

## 6. Champs "autre" et commentaires

Les XLSForm contiennent souvent des choix "Autre, préciser" ou des champs de commentaires. Ces champs peuvent être déterminants pour comprendre une anomalie.

**Impact utilisateur** : si ces compléments sont masqués ou mal placés, la fiche perd une partie de son sens métier.

**Impact maintenabilité** : il faut pouvoir relier un champ complémentaire à son champ principal.

## 7. Données géographiques

Les champs `geopoint`, latitude, longitude ou géométries doivent être affichés avec une mini-carte Leaflet, éventuellement accompagnée d'un lien vers la carte principale.

**Impact utilisateur** : une coordonnée brute est peu exploitable. Une mini-carte donne immédiatement le contexte spatial.

**Impact maintenabilité** : il faut normaliser les formats GPS Kobo et prévoir le cas des coordonnées absentes ou invalides.

## 8. Photos et pièces jointes

Les photos collectées peuvent être stockées sur Kobo, puis déplacées ou synchronisées vers Wasabi. L'affichage doit utiliser des URL signées temporaires, avec gestion des droits et des erreurs de chargement.

**Impact utilisateur** : les photos sont souvent des preuves terrain. Leur absence ou leur lenteur nuit à la capacité de contrôle.

**Impact maintenabilité** : les liens directs permanents sont à éviter. Il faut passer par un service applicatif qui génère des URL signées.

## 9. Performance et chargement progressif

Une fiche peut contenir beaucoup de champs et plusieurs photos. Charger toutes les images en pleine résolution dès l'ouverture peut ralentir fortement l'interface.

**Impact utilisateur** : l'ouverture de la fiche devient lente, surtout sur mobile ou connexion instable.

**Impact maintenabilité** : il faut prévoir miniatures, lazy loading, pagination des répétitions et limites raisonnables.

## 10. Longueur des textes

Certains commentaires ou réponses libres peuvent être longs. Ils doivent être lisibles sans casser la mise en page.

**Impact utilisateur** : les textes longs peuvent masquer les informations prioritaires.

**Impact maintenabilité** : il faut standardiser les composants d'affichage : extrait court, dépliage, bloc commentaire.

## 11. Données manquantes ou incohérentes

Les soumissions peuvent contenir des champs absents, des valeurs nulles, des formats inattendus ou des erreurs issues du terrain.

**Impact utilisateur** : une fiche qui affiche des erreurs brutes donne une impression d'instabilité.

**Impact maintenabilité** : le moteur de rendu doit prévoir des valeurs de remplacement comme "Non renseigné", "Invalide" ou "Non applicable".

## 12. Multilingue

G2M dispose déjà d'un mécanisme i18n. Les libellés du XLSForm peuvent aussi exister en plusieurs langues. Il faut choisir une stratégie de priorité entre les labels du formulaire et les locales applicatives.

**Impact utilisateur** : la fiche doit rester cohérente avec la langue de travail choisie dans l'application.

**Impact maintenabilité** : le mapping doit pouvoir accueillir plusieurs libellés ou pointer vers des clés i18n.

## 13. Sécurité et confidentialité

Certaines réponses peuvent être sensibles : identité d'enquêteur, coordonnées, photos, commentaires, informations personnelles. La fiche doit respecter les droits d'accès de l'utilisateur.

**Impact utilisateur** : les utilisateurs ne doivent voir que les informations nécessaires à leur rôle.

**Impact maintenabilité** : la sécurité doit être gérée côté serveur, pas uniquement par masquage frontend.

## 14. Évolution des formulaires

Les formulaires Kobo peuvent évoluer au fil des missions. Des champs peuvent être ajoutés, renommés ou supprimés.

**Impact utilisateur** : une fiche cassée après évolution du XLSForm réduit la confiance dans l'application.

**Impact maintenabilité** : il faut versionner ou dater les mappings utilisés pour interpréter les soumissions.

## Étape 2 - Tableau comparatif de trois solutions standards

| <b><i>Solution</i></b>                              | <b><i>Description</i></b>                                                                                                        | <b><i>Avantages</i></b>                                                                                                                                                           | <b><i>Inconvénients</i></b>                                                                                                                                                          | <b><i>Compatibilité avec G2M</i></b>                                                                                 |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Génération de vue HTML statique côté serveur</b> | Express assemble la soumission, le mapping XLSForm et les URL de médias, puis rend une page EJS ou Pug.                          | Très compatible avec l'existant ; contrôle complet du rendu ; sécurité côté serveur ; facile à intégrer dans les routes G2M ; bon pour le SEO interne et l'impression PDF future. | Moins interactif si aucun JavaScript complémentaire ; chaque changement d'ergonomie demande une modification de template ; nécessite une bonne structuration du mapping.             | Très forte. C'est l'approche la plus naturelle pour Node.js, Express, EJS, SQLite et l'architecture actuelle de G2M. |
| <b>Outil no-code ou low-code</b>                    | Utilisation d'un outil comme Budibase, Appsmith ou un dashboard externe pour construire des vues à partir des données.           | Mise en place rapide pour des prototypes ; composants prêts à l'emploi ; filtres et tableaux intégrés ; utile pour des administrateurs techniques.                                | Ajoute une plateforme supplémentaire ; intégration sécurité/JWT plus complexe ; dépendance externe ; personnalisation métier parfois limitée ; cohérence UI avec G2M plus difficile. | Moyenne. Possible pour un back-office parallèle, mais moins adapté à une intégration fine dans l'interface G2M.      |
| <b>Génération côté client</b>                       | Une API Express renvoie la soumission normalisée et un mapping ; JavaScript construit dynamiquement la fiche dans le navigateur. | Très interactif ; peut rafraîchir certaines sections sans recharger la page ; bon pour onglets dynamiques, mini-cartes et galeries photos ; séparation API/rendu.                 | Plus de logique côté frontend ; sécurité à renforcer côté API ; risque de duplication avec EJS ; nécessite une gestion robuste des états de chargement.                              | Forte. Compatible avec G2M si l'on reste en vanilla JS modulaire, mais plus complexe qu'un rendu serveur simple.     |

## Étape 3 - Proposition d'une approche maison efficace

### Principe général

L'approche recommandée est une solution maison hybride, centrée sur le serveur :

1. Express reçoit une demande de fiche pour une soumission donnée.
2. Le contrôleur récupère la soumission dans SQLite.
3. Le service de visualisation charge le mapping associé au formulaire ou à la mission.
4. Le service transforme le JSON brut Kobo en sections métier.
5. Le serveur génère les URL signées Wasabi pour les photos autorisées.
6. La page EJS affiche une fiche décisionnelle structurée.
7. Un JavaScript léger initialise les composants interactifs : mini-carte Leaflet, galerie photos, dépliage des sections longues.

Cette solution respecte l'architecture actuelle, évite une refonte frontend, et garde la sécurité principale côté serveur.

### Route cible possible

Le module peut être intégré dans G2M via une route de détail :

- `/soumissions/:id/detail`
- ou `/cartographie/sites/:id`
- ou un onglet "Détail enquête" dans la fiche site déjà affichée depuis la carte.

La même logique peut servir à une vue HTML, une impression PDF future ou une exportation contrôlée.

### Structure recommandée du mapping

Le mapping peut être stocké d'abord sous forme JSON dans `config/forms/` ou `data/form-mappings/`, puis migré plus tard en base.

Exemple de structure :

```
{
  "formUid": "aBcDeF123",
  "version": "2026-06-06",
  "title": "Fiche d'identification du site",
  "primaryKey": "_id",
  "sections": [
    {
      "id": "identification",
      "label": "Identification",
      "order": 1,
      "fields": [
        {
          "name": "site/code_site",
          "label": "Code du site",
          "type": "text",
          "priority": "high"
        }
      ]
    }
  ]
}
```

```

        {
            "name": "site/nom_site",
            "label": "Nom du site",
            "type": "text",
            "priority": "high"
        },
        {
            "name": "site/statut",
            "label": "Statut de contrôle",
            "type": "choice",
            "choices": {
                "valide": "Validé",
                "a_verifier": "À vérifier",
                "rejete": "Rejeté"
            },
            "badge": true
        }
    ]
},
{
    "id": "localisation",
    "label": "Localisation",
    "order": 2,
    "fields": [
        {
            "name": "gps_site",
            "label": "Coordonnées GPS",
            "type": "geopoint",
            "map": true
        }
    ]
},
{
    "id": "preuves",
    "label": "Photos et preuves",
    "order": 3,
    "fields": [
        {
            "name": "photo_facade",
            "label": "Photo de façade",
            "type": "image",
            "storage": "wasabi"
        }
    ]
},
{
    "id": "equipements",
    "label": "Équipements observés",
    "type": "repeat",
    "name": "equipements",
    "display": "table",
    "fields": [
        {
            "name": "equipement_type",
            "label": "Type",
            "type": "choice"
        },
        {
            "name": "etat",
            "label": "État",
            "type": "choice"
        }
    ]
}

```

```

    },
    {
      "name": "commentaire",
      "label": "Commentaire",
      "type": "long_text"
    }
  ]
}
]
}

```

## Organisation du code

Une structure pragmatique pourrait être :

```

services/
  submissionViewService.js
  formMappingService.js
  mediaUrlService.js
controllers/
  submissionDetailController.js
routes/
  submissionDetailRoutes.js
views/
  submissions/
    detail.ejs
  partials/
    field-value.ejs
    repeat-table.ejs
    photo-gallery.ejs
    mini-map.ejs
public/js/
  submission-detail.js
config/forms/
  kobo-form-aBcDeF123.json

```

## Transformation des données

Le service `submissionViewService` doit produire un objet prêt à être rendu, par exemple :

```

{
  "title": "Fiche site - CI-ABJ-001",
  "badges": [
    {
      "label": "Validé",
      "tone": "success"
    }
  ]
}

```

```

    },
    {
      "label": "GPS disponible",
      "tone": "info"
    }
  ],
  "sections": [
    {
      "label": "Identification",
      "fields": [
        {
          "label": "Code du site",
          "type": "text",
          "value": "CI-ABJ-001"
        },
        {
          "label": "Statut de contrôle",
          "type": "choice",
          "value": "Validé",
          "tone": "success"
        }
      ]
    }
  ]
}

```

La vue EJS ne doit pas connaître les noms techniques Kobo. Elle doit recevoir une structure déjà interprétée.

## Gestion des photos Wasabi

Les photos doivent être servies par URL signées temporaires :

1. la soumission contient le nom de fichier ou la référence média ;
2. G2M retrouve l'objet Wasabi correspondant ;
3. le serveur génère une URL signée à durée courte, par exemple 5 à 15 minutes ;
4. la fiche affiche une miniature avec `loading="lazy"` ;
5. un clic ouvre l'image en grand dans une modale.

Il est préférable de ne jamais stocker d'URL signée en base. On stocke seulement la clé objet Wasabi, puis on signe à la demande.

Exemple de structure de champ photo transformé :

```

{
  "label": "Photo de façade",
  "type": "image",
  "value": {
    "fileName": "photo_facade_001.jpg",
    "thumbnailUrl": "/media/submissions/123/photo_facade?size=thumb",
    "signedUrl": "/media/submissions/123/photo_facade?size=full"
  }
}

```



## Gestion des groupes répétés

Les répétitions doivent être converties en sous-sections. Deux rendus sont recommandés :

- tableau compact pour les répétitions homogènes et courtes ;
- cartes répétées pour les blocs riches contenant commentaires, photos ou statuts.

Exemple :

```
{
  "label": "Équipements observés",
  "type": "repeat",
  "display": "table",
  "rows": [
    [
      {
        "label": "Type",
        "value": "Antenne"
      },
      {
        "label": "État",
        "value": "Bon"
      }
    ],
    [
      {
        "label": "Type",
        "value": "Batterie"
      },
      {
        "label": "État",
        "value": "À remplacer"
      }
    ]
  ]
}
```

## Affichage géographique

Pour les champs `geopoint`, G2M doit :

- parser les coordonnées Kobo, souvent au format **latitude longitude altitude accuracy** ;
- afficher **latitude** et **longitude** formatées ;
- afficher une mini-carte Leaflet dans la fiche ;
- proposer un lien "Voir sur la carte SIG" vers la carte principale ;
- afficher un état clair si les coordonnées sont absentes ou invalides.

La mini-carte ne doit pas charger tous les points. Elle doit seulement afficher le point de la soumission courante.

## Style orienté décision

La fiche doit prioriser la décision, pas la restitution exhaustive brute.

Choix recommandés :

- en-tête avec nom/code du site, statut, mission, région, date de soumission ;
- badges visuels pour les statuts : validé, à vérifier, rejeté, anomalie ;
- sections repliables pour les détails secondaires ;
- grille de champs lisible, deux colonnes sur desktop, une colonne sur mobile ;
- blocs "À surveiller" ou "Anomalies" en haut de fiche ;
- galerie photo avec miniatures ;
- mini-carte encadrée mais compacte ;
- bouton d'accès au JSON brut réservé aux profils techniques.

## Endpoint Express illustratif

```
const express = require("express");
const router = express.Router();
const SoumissionCollecte = require("../models/SoumissionCollecte");
const {
  buildSubmissionView
} = require("../services/submissionViewService");
router.get("/soumissions/:id/detail", async (req, res, next) => {
  try {
    const submission = SoumissionCollecte.findById(req.params.id);
    if (!submission) {
      return res.status(404).render("errors/404", {
        title: req.t("errors.notFound.title")
      });
    }
    const viewModel = await buildSubmissionView({
      submission,
      locale: req.locale,
      user: req.user
    });
    res.render("submissions/detail", {
      title: viewModel.title,
      viewModel
    });
  } catch (error) {
    next(error);
  }
});
module.exports = router;
```

## Service de transformation illustratif

```
const {loadMappingForSubmission } = require("../formMappingService");
const {valueAtPath } = require("../koboPayloadMapper");
const { signSubmissionMediaUrl } = require("../mediaUrlService");
async function buildSubmissionView({ submission, locale, user}) {
  const rawData = JSON.parse(submission.raw_data_json);
  const mapping = loadMappingForSubmission(submission);
  const sections = await Promise.all(mapping.sections.map(async (section) => {
    if (section.type === "repeat") {
      return buildRepeatSection(section, rawData);
    }
  }
  const fields = await Promise.all(section.fields.map((field) => buildFieldView(field, rawData,
    { locale, user, submission })))));
```

```

    return {
      id: section.id,
      label: section.label,
      fields: fields.filter(Boolean)
    };
  });
}

    return {
      title: mapping.title,
      submissionId: submission.id,
      sections,
      rawDataAvailable: user?.role === "admin" || user?.role === "specialiste_gis"
    };
}
}
async function buildFieldView(field, rawData, context) {
  const rawValue = valueAtPath(rawData, field.name);
  if (rawValue === undefined || rawValue === null || rawValue === "") {
    return {
      label: field.label,
      type: field.type,
      value: "Non renseigné",
      empty: true
    };
  }
  if (field.type === "choice") {
    return {
      label: field.label,
      type: "choice",
      value: field.choices?.[rawValue] || rawValue
    };
  }
  if (field.type === "image") {
    return {
      label: field.label,
      type: "image",
      value: {
        fileName: rawValue,
        url: await signSubmissionMediaUrl(context.submission.id, rawValue, context.user)
      }
    };
  }
  if (field.type === "geopoint") {
    return {
      label: field.label,
      type: "geopoint",
      value: parseKoboGeopoint(rawValue)
    };
  }
  return {
    label: field.label,
    type: field.type,
    value: rawValue
  };
}
}
module.exports = {
  buildSubmissionView
};

```

Template EJS simplifié

```

< % -include("../partials/header", { title }) %>
< section class="submission-detail">
  < header class="submission-detail-header">
    < div>
      < p class="eyebrow"> Détail enquête < /p>
      <h1><%= viewModel.title %></h1>
    < /div>
  </header>
  < % viewModel.sections.forEach((section)=> {
    % > < section class="panel submission-section">
      < h2>
        < %=section.label %>
        < /h2> <% if (section.type === "repeat") { %> <%= include("../partials / repeat
- table ", { section }) %> <% } else { %> <div class="
      submission - field - grid "> <% section.fields.forEach((field) => { %> <article class="
      submission - field < %= field.empty ? " is-empty" : "" %> "> <span class="
      submission - field - label "><%= field.label %></span> <% if (field.type === "
      image ") { %>  "
loading="
      lazy "> <% } else if (field.type === "
      geopoint ") { %> <div class="
      submission - mini - map " data-lat=" < %= field.value.latitude % > " data-lng=" < %=
field.value.longitude % > "></div> <% } else { %> <strong><%= field.value %></strong> <% } %>
</article> <% }) %> </div> <% } %> </section> <% }) %> </section>
      <script src=" / js / submission - detail.js "></script> <%= include("../ /
partials / footer ") %>

```

## CSS indicatif

```

.submission - detail {
  display: grid;
  gap: 18 px;
}

.submission - detail - header {
  align - items: center;
  display: flex;
  justify - content: space - between;
}

.submission - field - grid {
  display: grid;
  gap: 12 px;
  grid - template - columns: repeat(2, minmax(0, 1 fr));
}

.submission - field {
  background: #ffffff;
  border: 1 px solid var(--border);
  border - radius: 6 px;
  padding: 12 px;
}

.submission - field - label {
  color: var(--muted);
  display: block;
  font - size: var(--font - small);
  margin - bottom: 4 px;
}

```

```
.submission - field.is - empty strong {  
  color: var (--muted);  
  font - style: italic;  
}  
  
.submission - mini - map {  
  border: 1 px solid var (--border);  
  border - radius: 6 px;  
  height: 220 px;  
  overflow: hidden;  
}
```

## Recommandation finale

La meilleure approche pour G2M est une génération serveur via Express et EJS, alimentée par un fichier de mapping JSON issu ou inspiré du XLSForm, complétée par quelques composants JavaScript légers pour les mini-cartes, les galeries photos et les sections repliables.

Cette approche est pragmatique, car elle s'appuie sur l'existant. Elle est maintenable, car les libellés, types et groupes sont externalisés dans un mapping. Elle est évolutive, car le même modèle pourra être utilisé plus tard avec PostgreSQL/PostGIS, avec un stockage de mapping en base et une génération de rapports PDF.

La première version devrait viser les fonctionnalités suivantes :

1. fiche décisionnelle par soumission ;
2. mapping JSON minimal `name`, `label`, `section`, `type` ;
3. rendu EJS par sections ;
4. affichage des choix lisibles ;
5. mini-carte pour le GPS ;
6. galerie simple pour les photos Wasabi ;
7. bouton réservé aux profils techniques pour voir le JSON brut.

Cette trajectoire permet de livrer rapidement une valeur métier forte, tout en préparant une architecture de visualisation robuste pour les évolutions futures de G2M.